

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

LABORATORIO 5

SQL (2/2)

Departamento de Industrias

ICN-292 — Sistemas de Información para la Gestión

Paralelo: 101

Profesor: J. Saez

Alumno: Carlos Martínez González

Rol: 202466517-0

URL Repositorio:https://github.com/emg46408-cmd/ICN292-Lab5-MartinezGonzalez-Carlos.git

Entregable 2: Consultas

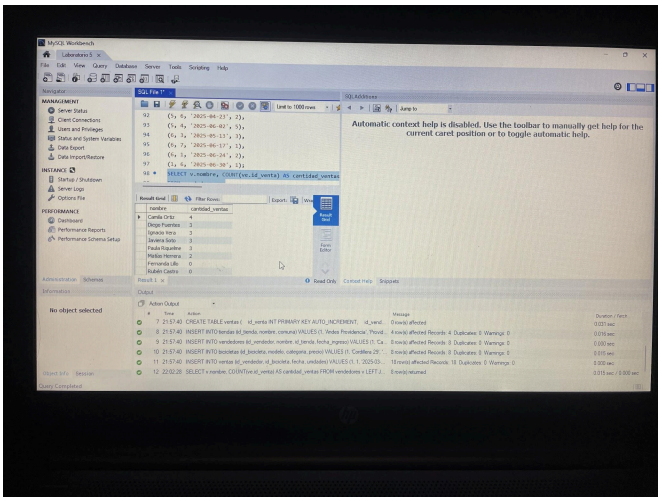
Consulta 1

Objetivo: Liste todos los vendedores con la cantidad de ventas que registraron, incluidos los que no registraron ninguna. Ordene de mayor a menor cantidad y, a igualdad, por nombre. Los que no tienen ventas deben mostrar cero, no quedar en blanco.

Código SQL ejecutado:

```
SELECT v.nombre, COUNT(ve.id_venta) AS cantidad_ventas
FROM vendedores v
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY cantidad_ventas DESC, v.nombre ASC;
```

Resultado:



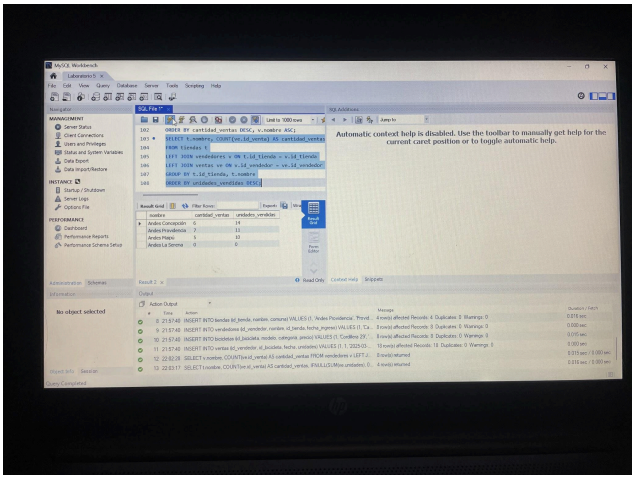
Consulta 2

Objetivo: Para cada tienda muestre su nombre, cuántas ventas se registraron y cuántas unidades se vendieron, incluidas las tiendas que no tienen ningún vendedor. Ordene por unidades de mayor a menor.

Código SQL ejecutado:

```
SELECT t.nombre, COUNT(ve.id_venta) AS cantidad_ventas, IFNULL(SUM(ve.unidades), 0) AS unidades_vendidas
FROM tiendas t
LEFT JOIN vendedores v ON t.id_tienda = v.id_tienda
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades_vendidas DESC;
```

Resultado:



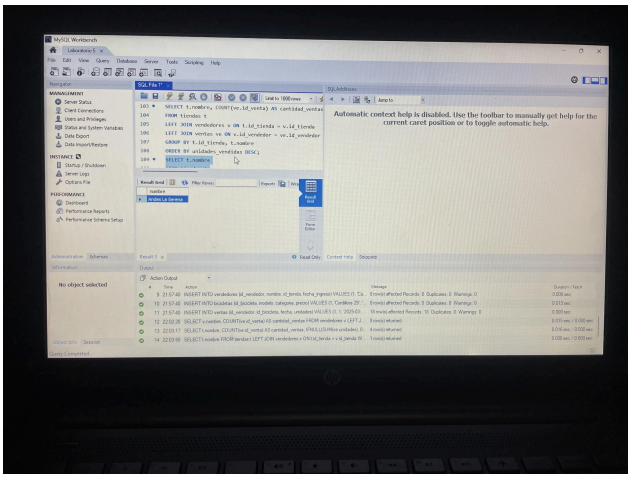
Consulta 3

Objetivo: Liste las tiendas que no tienen ningún vendedor asignado.

Código SQL ejecutado:

```
SELECT t.nombre
FROM tiendas t
LEFT JOIN vendedores v ON t.id_tienda = v.id_tienda
WHERE v.id_vendedor IS NULL;
```

Resultado:



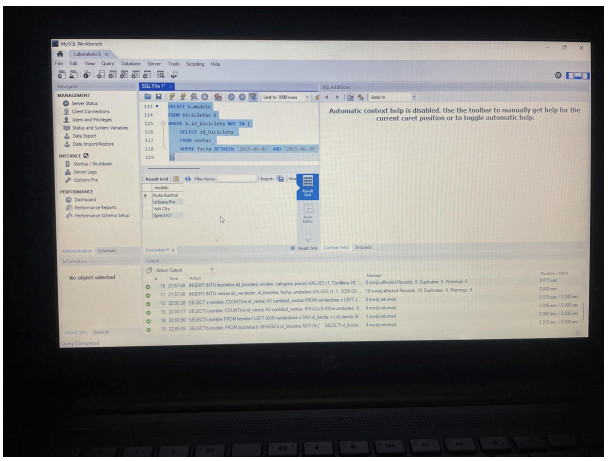
Consulta 4

Objetivo: Liste los modelos que no se vendieron ni una sola vez en junio de 2025. Explique en una línea en qué parte de la consulta puso la condición del mes y qué habría devuelto si la hubiera puesto en la otra.

Código SQL ejecutado:

```
SELECT b.modelo
FROM bicicletas b
WHERE b.id_bicicleta NOT IN (
  SELECT id_bicicleta
  FROM ventas
  WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30'
);
```

Resultado:



Explicación: La condición del mes se colocó en el WHERE de una subconsulta para identificar exactamente qué bicicletas sí tuvieron ventas ese mes y luego excluirlas; si se hubiera puesto como condición directa en un LEFT JOIN o en el WHERE de la consulta principal, habría filtrado incorrectamente las ventas de otros meses y devuelto datos erróneos al mezclar modelos.

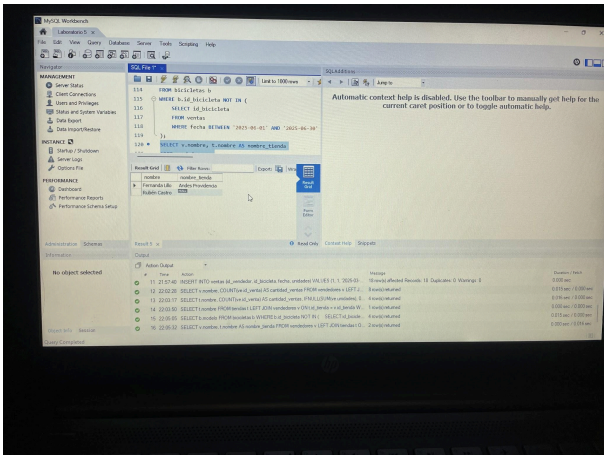
Consulta 5

Objetivo: Liste los vendedores que no han registrado ninguna venta, mostrando también el nombre de su tienda. Ninguno de ellos puede quedar fuera del resultado.

Código SQL ejecutado:

```
SELECT v.nombre, t.nombre AS nombre_tienda
FROM vendedores v
LEFT JOIN tiendas t ON v.id_tienda = t.id_tienda
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
WHERE ve.id_venta IS NULL;
```

Resultado:



Consulta 6

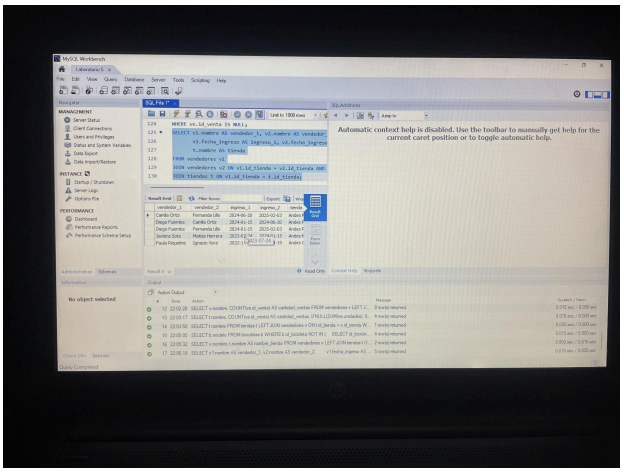
Objetivo: Liste los pares de vendedores que trabajan en la misma tienda, poniendo primero al que ingresó antes. Muestre los dos nombres, las dos fechas de ingreso y la tienda. Cada par debe aparecer una sola vez.

Código SQL ejecutado:

```
SELECT v1.nombre AS vendedor_1, v2.nombre AS vendedor_2,
    v1.fecha_ingreso AS ingreso_1, v2.fecha_ingreso AS ingreso_2,
    t.nombre AS tienda
```

```
FROM vendedores v1
JOIN vendedores v2 ON v1.id_tienda = v2.id_tienda AND v1.fecha_ingreso < v2.fecha_ingreso
JOIN tiendas t ON v1.id_tienda = t.id_tienda;
```

Resultado:



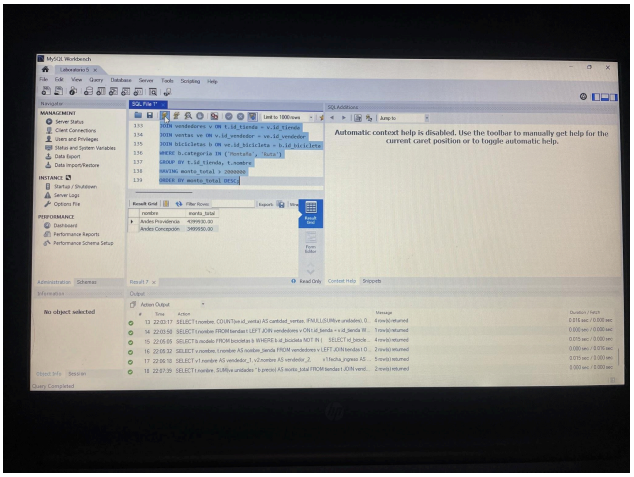
Consulta 7

Objetivo: Para cada tienda, calcule el monto total vendido en bicicletas de las categorías 'Montaña' y 'Ruta', dejando solo las tiendas que superan los 2.000.000. Ordene de mayor a menor.

Código SQL ejecutado:

```
SELECT t.nombre, SUM(ve.unidades * b.precio) AS monto_total
FROM tiendas t
JOIN vendedores v ON t.id_tienda = v.id_tienda
JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
JOIN bicicletas b ON ve.id_bicicleta = b.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING monto_total > 2000000
ORDER BY monto_total DESC;
```

Resultado:

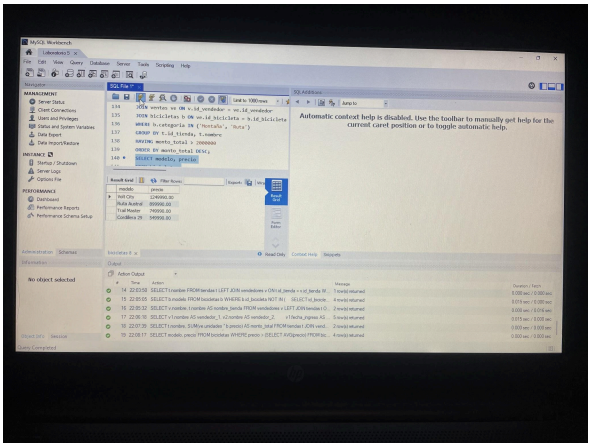


Consulta 8

Objetivo: Liste el modelo y el precio de las bicicletas cuyo precio supera el promedio del catálogo, de mayor a menor. Indique en el informe el valor de ese promedio y de dónde lo obtuvo.

Código SQL ejecutado:
SELECT modelo, precio
FROM bicicletas
WHERE precio > (SELECT AVG(precio) FROM bicicletas)
ORDER BY precio DESC;

Resultado:



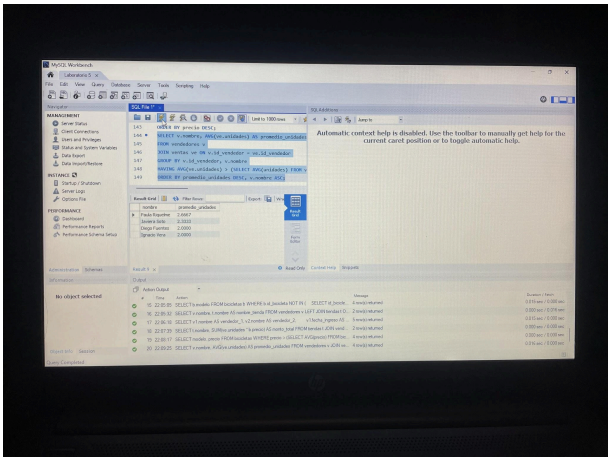
Explicación : El promedio del catálogo es de $657.490 y se obtuvo internamente de forma dinámica utilizando la subconsulta (SELECT AVG(precio) FROM bicicletas) dentro de la cláusula WHERE.

Consulta 9

Objetivo: Liste los vendedores cuyo promedio de unidades por venta supera el promedio de unidades de todas las ventas de la cadena. Muestre el nombre y su promedio, de mayor a menor y, a igualdad, por nombre.

Código SQL ejecutado:
SELECT v.nombre, AVG(ve.unidades) AS promedio_unidades
FROM vendedores v
JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
HAVING AVG(ve.unidades) > (SELECT AVG(unidades) FROM ventas)
ORDER BY promedio_unidades DESC, v.nombre ASC;

Resultado:



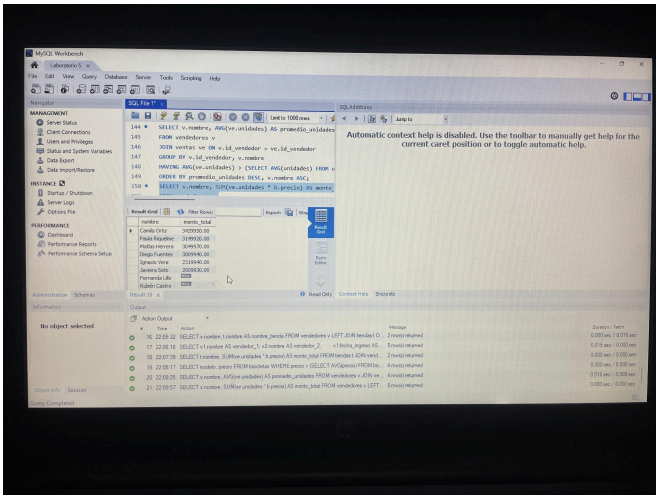
Consulta 10

Objetivo: Liste todos los vendedores con el monto total que vendieron, incluidos los que no vendieron nada, de mayor a menor monto. Explique en una línea por qué algunos vendedores no muestran cero y qué tendría que agregar a la consulta para que lo mostraran.

Código SQL ejecutado:

```
SELECT v.nombre, SUM(ve.unidades * b.precio) AS monto_total
FROM vendedores v
LEFT JOIN ventas ve ON v.id_vendedor = ve.id_vendedor
LEFT JOIN bicicletas b ON ve.id_bicicleta = b.id_bicicleta
GROUP BY v.id_vendedor, v.nombre
ORDER BY monto_total DESC;
```

Resultado:



Explicación: Al hacer un LEFT JOIN, los vendedores sin registros de venta generan campos vacíos, por lo que la función SUM() retorna NULL; para que mostraran un cero, habría que envolver la operación en una función de manejo de nulos en el SELECT, como por ejemplo IFNULL(SUM(ve.unidades * b.precio), 0).